DEVOPS FINAL CAPSTONE · SESSION 21

StockPilot

Inventory management, shipped through a full DevOps pipeline

Kiran · Roll No. 10446

github.com/Kirann1973/stockpilot-devops-capstone

02

Project overview

What StockPilot does and who it is for

Problem Small stores and offices track stock in spreadsheets and run out of items without warning.

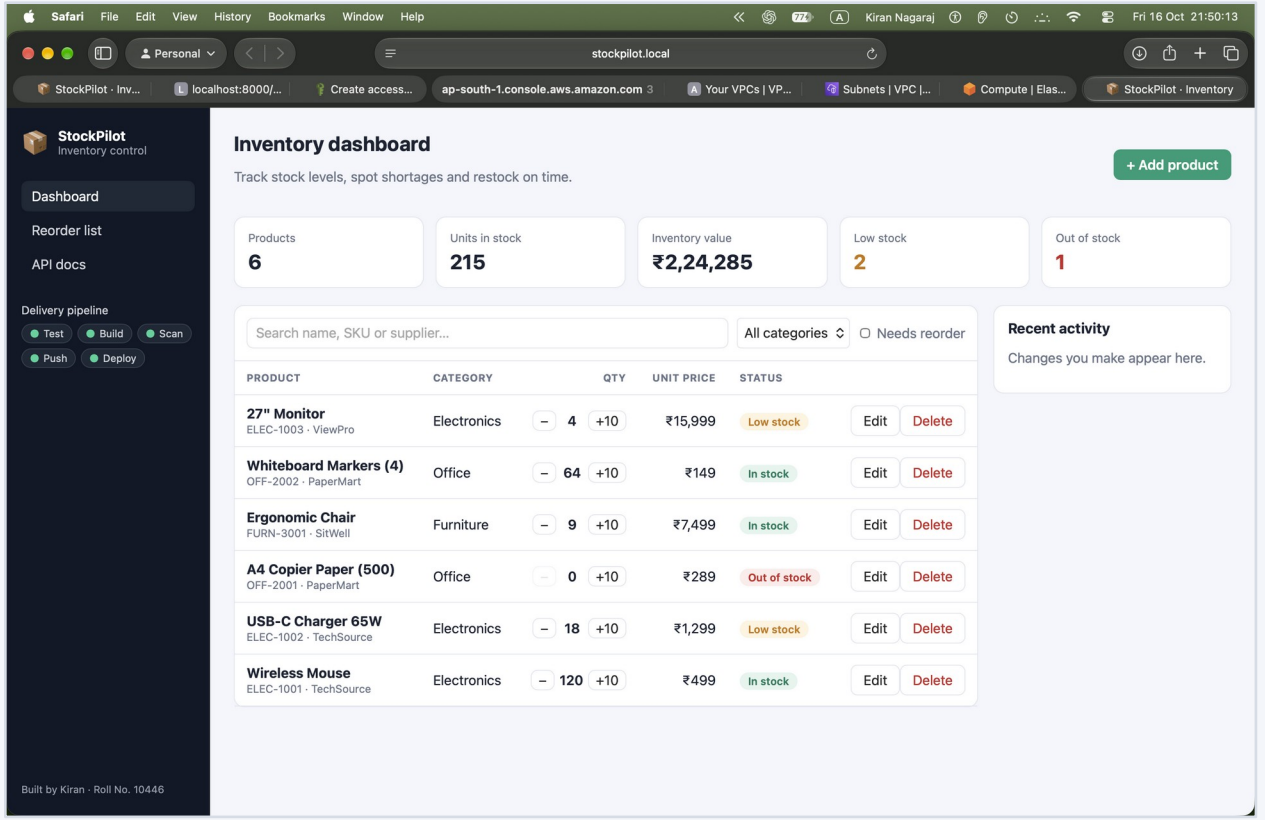
Solution A web dashboard: product catalogue with SKU, category, supplier and price.

Stock status In stock, Low stock (at or below the reorder level), Out of stock.

One-click updates Receive or ship stock with +10 / − buttons.

KPIs Product count, units on hand, inventory value, low and out-of-stock counts.

API FastAPI REST: CRUD on /api/products, stats, stock adjust, /health, /ready, /metrics.



03

Tech stack

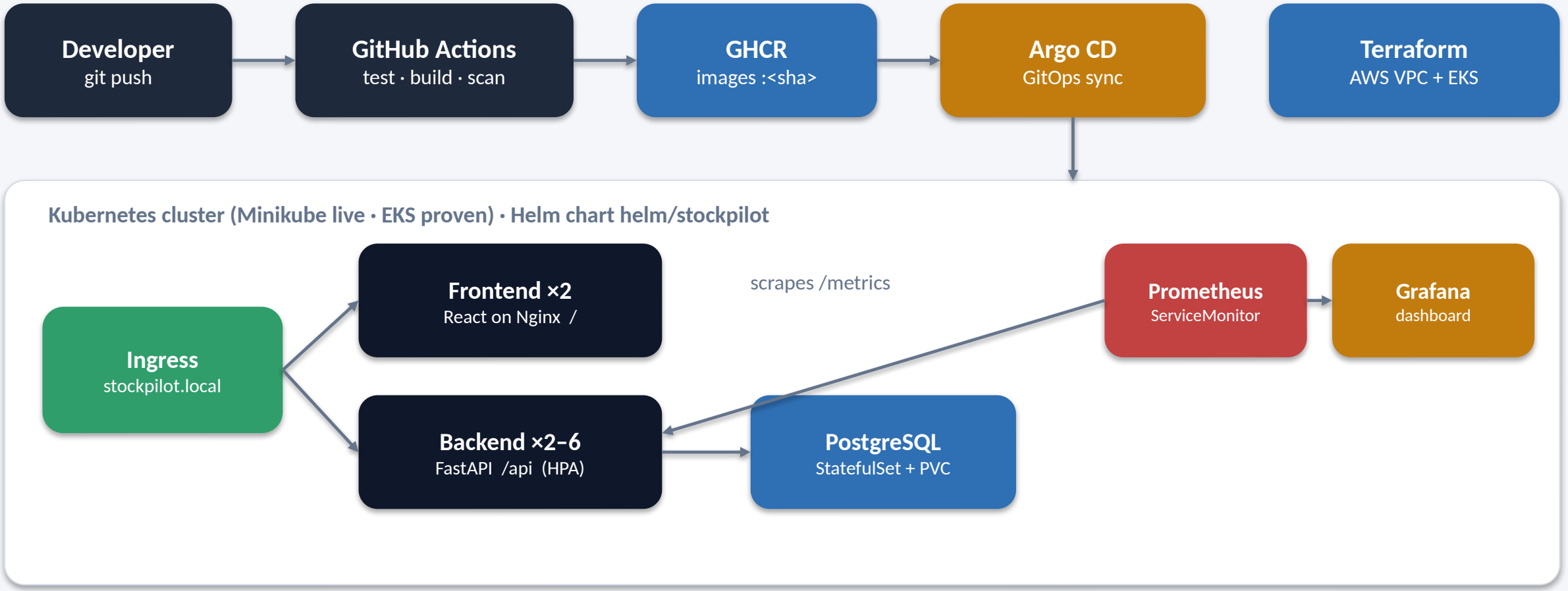
Every layer from the browser to the cloud

Frontend	React 18 + Vite, responsive CSS, Nginx (unprivileged)
Backend	FastAPI, SQLAlchemy 2, Alembic migrations, Pydantic
Database	PostgreSQL 16
Testing	Pytest, 14 tests on an isolated SQLite database
Containers	Docker (non-root images), Docker Compose
CI/CD	GitHub Actions → GitHub Container Registry (tag = commit SHA)
Security	Trivy image scan, fails on HIGH / CRITICAL
Infrastructure	Terraform: AWS VPC + EKS 1.34 in ap-south-1
Kubernetes	Helm chart, NGINX Ingress, HPA, Argo CD (GitOps)
Observability	Prometheus (kube-prometheus-stack) + Grafana

04

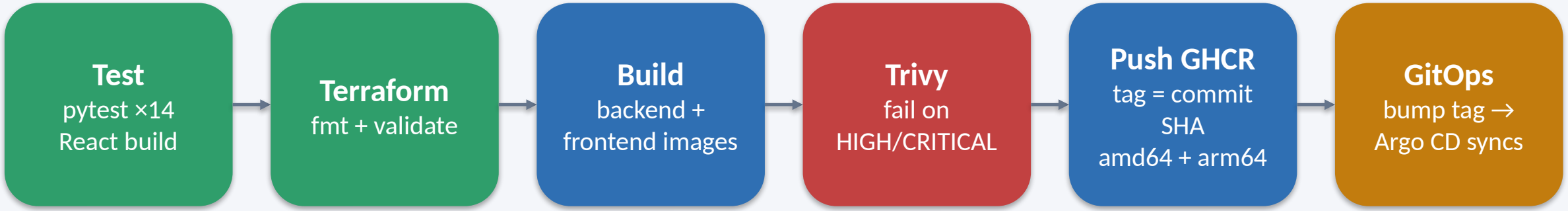
Architecture

From a git push to a monitored Kubernetes cluster



CI/CD pipeline

.github/workflows/ci-cd.yml runs on every push to main



Quality gate first A failing test stops the pipeline, so no broken image is ever built or pushed.

Traceability Every image is tagged with the Git commit SHA, never "latest". Commit → image → pod.

Multi-arch Images are published for amd64 (EKS) and arm64 (Apple Silicon Minikube).

Pull-based deploy CI only commits the new tag to Git. Argo CD pulls it into the cluster, so CI holds no cluster credentials.

DevSecOps: Trivy security gate

The scan really blocked a vulnerable image

Run #1: BLOCKED
7 HIGH CVEs in the backend image

→

Fix
Upgrade FastAPI / Starlette, remove pip + setuptools from the runtime image

→

Runs #2–#4: CLEAN
0 HIGH / CRITICAL, images pushed

What is scanned Both images: OS packages (Debian, Alpine) and Python libraries, against public CVE databases.

Gate --severity HIGH,CRITICAL --ignore-unfixed --exit-code 1. On a finding the job fails and the push step is skipped.

Example CVE urllib3 2.7.0 (CVE-2026-97687): traffic interception via HTTPS proxy TLS configuration override. Fixed in 2.8.0.

What "clean" means No known fixable high-risk CVEs at scan time. It is one layer; SAST, secret scanning and runtime security are separate.

07

Infrastructure as Code: Terraform on AWS

terraform/ · region ap-south-1 (Mumbai)

VPC 10.0.0.0/16 with 2 public + 2 private subnets across 2 AZs, one NAT gateway.

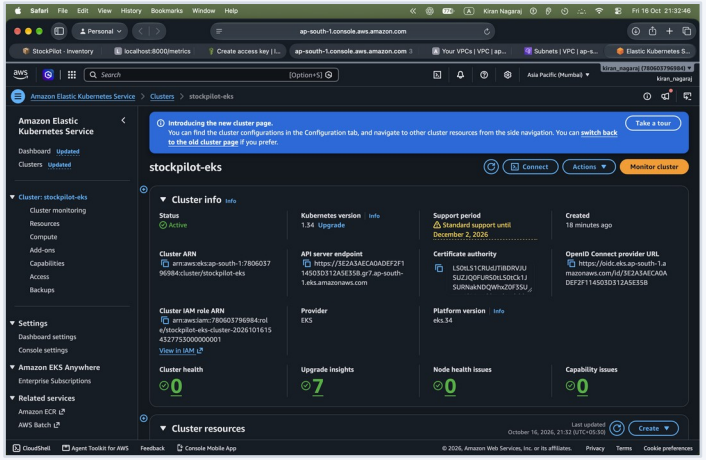
EKS 1.34 Managed control plane, node group "main" with 2 × t3.medium in private subnets.

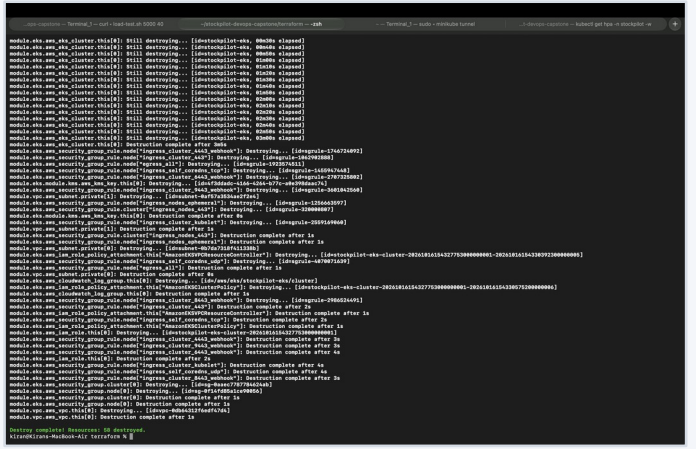
Modules terraform-aws-modules/vpc 5.21 and /eks 20.37, pinned versions.

Lifecycle init → plan (58 to add) → apply → verified 2 nodes Ready → destroy (58 destroyed).

Safety No credentials in Git: terraform.tfvars.example only, *.tfvars and state are git-ignored.

Cost About $0.25–0.30 per hour, destroyed the same day.





Kubernetes, Helm and GitOps

One Helm chart, deployed by Argo CD

Helm chart helm/stockpilot: Deployments, ClusterIP Services, Ingress, HPA, Postgres StatefulSet + PVC, Secret.

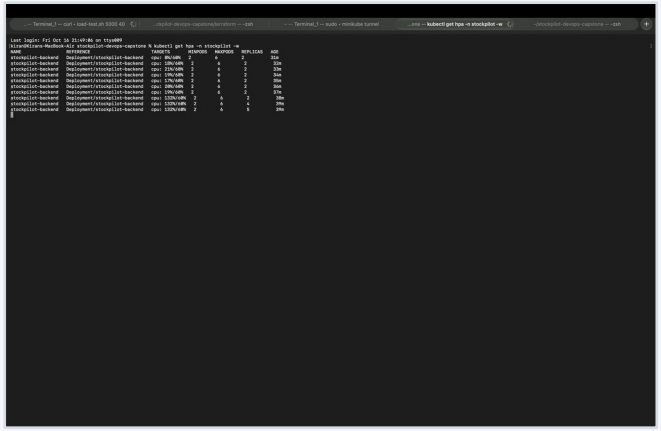
Replicas Frontend ×2, backend ×2 (HPA scales up to 6 at 60% CPU).

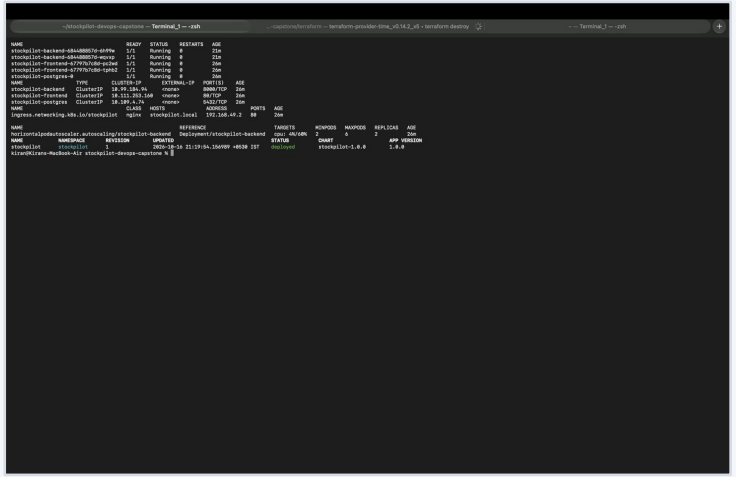
Ingress / → frontend, /api → backend on stockpilot.local (NGINX Ingress).

Probes /health for liveness, /ready checks the database before taking traffic.

HPA proven Under load, CPU hit 132% of target and the backend scaled 2 → 4 → 6 pods.

Argo CD Watches main; CI commits the new image SHA, Argo CD syncs the cluster automatically.





09

Observability: Prometheus + Grafana

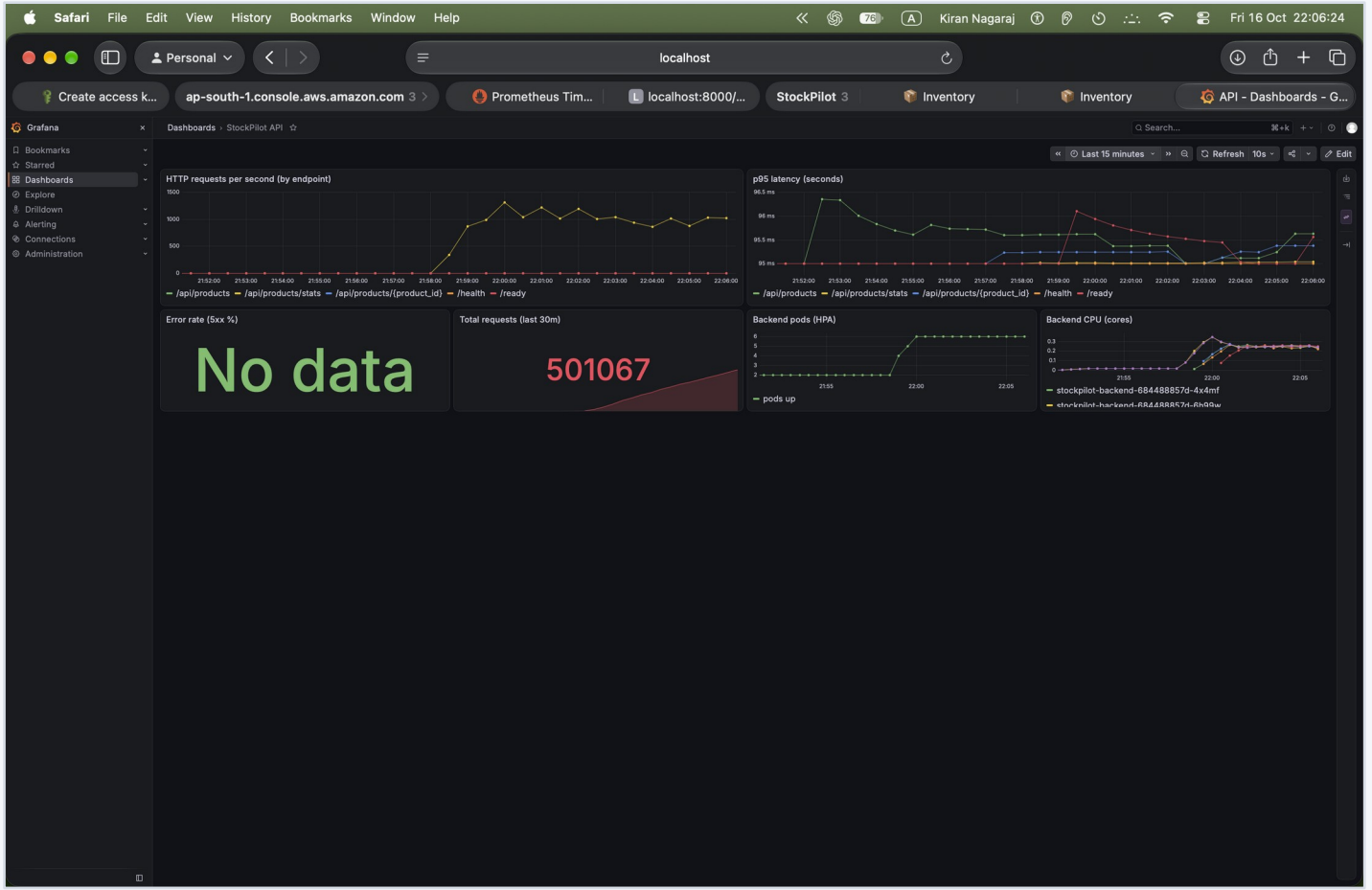
kube-prometheus-stack, ServiceMonitor, provisioned dashboard

/metrics prometheus-fastapi-instrumentator exposes request counts and latency histograms.

Prometheus ServiceMonitor scrapes every backend pod: 6/6 targets UP during the load test.

Grafana "StockPilot API" dashboard: requests/s per endpoint, p95 latency, error rate, total requests, HPA pods, CPU per pod.

Result ~1,300 req/s peak, ~500k requests, 0 server errors.



Key learnings

What building this taught me

Gates beat good intentions

Tests and Trivy run before any push, so a bad commit cannot reach the registry. Run #1 proved it.

Tag by commit SHA

Every running pod traces back to one exact commit, and rollback is just an older tag.

Infrastructure is code

58 AWS resources created and destroyed from the same files, repeatable and reviewable.

Platforms matter

Apple Silicon needed multi-arch images; EKS versions have support windows that change cost.

Pull beats push for deploys

Argo CD keeps the cluster equal to Git; CI never needs cluster credentials.

Measure, then scale

The HPA only scaled once real load pushed CPU over target; Grafana showed it happening.

Thank you

Live walkthrough next: code → pipeline → Argo CD → Kubernetes → Prometheus → Grafana

Kiran · Roll No. 10446

github.com/Kirann1973/stockpilot-devops-capstone